LIMO Milestone 2 – Summary and Possible Gaps

CS3520 LIMO Group Project · Team Data-Academia · prepared 9 November 2026

Basis: the repository zip (limo-Data-Academia-main-1.zip). The zip does not contain the official M2 brief, so "what may be missing" is judged against the M1 ISA document, the M2 design file (docs/CONTROL_UNIT_AND_ALU_LIMO.md) and what the code actually does. Check the gaps below against your M2 brief before acting on them.

1. What M2 delivers

M2 turns the M1 ISA into a hardware design for the decode stage, control unit and ALU. It is written in docs/CONTROL_UNIT_AND_ALU_LIMO.md and backed by C++ models in src/.

Instruction decode. Fields are opcode [6:0], rd [10:7], funct3 [14:12], rs1 [18:15], rs2 [23:20] and funct7 [31:25]. Register numbers are 4 bits, so bits 11, 19 and 24 are reserved and must be 0, except where they carry immediate bits (S, B and I-shift). The doc lists the ten legal patterns and the immediate generator for I, S and B formats.

Illegal words. Anything that is not one of the ten patterns, or has a reserved bit set, decodes as a NOP with every control signal at 0. Data-address bits [1:0] are ignored. Both are implementation choices inside what the ISA calls "undefined".

Main control unit. One table gives ALUSrc, MemToReg, RegWrite, MemRead, MemWrite, Branch, ALUOp and ImmSel for each instruction class. Signals are grouped by the stage that uses them (EX, MEM, WB). Branches resolve in EX with a separate target adder, and a taken branch flushes the two instructions behind it.

ALU and ALU control. The ALU is 32-bit and purely combinational, with five operations (ADD, SLL, OR, AND, SUB) and a single Zero status output. ALU_Control is {funct7[5]_eff, funct3}. ALUOp 11 exists so that bit 30 is ignored for eketsa; otherwise eketsa r3, r1, −1024 (0xC0008193) would run as tlosa.

Verification table. Fourteen test vectors with hex encodings and expected control values: all ten instructions, a negative-immediate eketsa, a negative branch offset, and two NOP cases (reserved bit set, and jal).

Code behind it. ControlUnit, InstructionDecoder, ALU, RegisterFile and Memory, plus a single-cycle CPU model and three smoke tests.

2. What I checked

- The three smoke tests (register/memory, M1 instruction, pipeline) compile and pass. The pipeline test's control-unit section covers all 14 vectors from the verification table.
- The single-cycle CPU model runs the three M1 sample programs to the expected final state: Program 1 in 18 instructions, Program 2 in 22 and Program 3 in 10, all matching the M1 document.

3. What may be missing

1. **No hazard or forwarding design.** Section 2.1 only describes the interface to a hazard unit (flush and stall bubbles, forwarding mux positions). The unit itself is not designed: no forwarding conditions, no load-use detection, no stall logic. M1 design-decision (d) promises a one-cycle load-use stall.
2. **No datapath or timing diagrams.** The doc covers control and ALU only. There is no block diagram, no pipeline-register contents table, and no specification of the register file, PC or memories.
3. **Decision-log entries not written.** The doc says the NOP-on-illegal and ignore-address-bits choices "can be recorded" as design-decision log entries. No such entries exist in the repo.
4. **Verification stops at decode.** ALU results and Zero are not tested against the table vectors, and none of the three M1 programs is stored as an example file or run automatically against its expected registers, memory and cycle count (30, 33 and 17 cycles in M1). Only three tests are registered in CMake.

5 **Code does not match the M1 spec in places.** Memory is one 1 MiB array shared by code and data; M1 specifies separate instruction and data memories and a 4 GB space, and the M1 programs store to Mem[0], where their own first instruction lives. The single-cycle CPU::step throws on illegal instructions and misaligned addresses, but the design says to treat them as NOP and to ignore address bits [1:0] (the pipeline does this correctly). ALU.cpp also implements XOR, SRL/SRA and SLT/SLTU, which are outside the ISA and the “five codes only” design.

6 **Repository housekeeping.** The README promises top-level tests/ and examples/ folders, but tests live in src/tests and the only examples are two PDFs. The README live link is still a placeholder, and there is no C++ command-line assembler in this version of the repo.

7 **The simulator itself.** The page in index.html is an assembler with static placeholder tables, and the pipeline lacks a hazard unit. See the separate document, *Why the LIMO Processor Simulator Does Not Work*.

4. Suggested order

Write the hazard-unit section of the design first, since it blocks both the pipeline and the M1 cycle counts. Then add the datapath diagram and decision-log entries. Then store the M1 programs as examples with expected results, and align Memory and CPU::step with the spec.